CineMetrics API — Technical Report

COMP3011 Coursework 1 | University of Leeds | March 2026

GitHub: https://github.com/Thach2r/movie-api

Deployment: https://yeThatcher.pythonanywhere.com

1. Technology Stack Justification

Python was selected as the primary programming language because the project focuses on building a data-driven web service that integrates multiple datasets and performs analytical queries. Python provides a mature ecosystem for data processing and backend development, which makes it particularly suitable for this type of application. Libraries such as pandas enable efficient data handling and transformation, while SQLAlchemy simplifies database interaction through an object-relational mapping (ORM) approach. Together, these tools allow the system to process historical datasets and interact with relational databases in a structured and maintainable way.

The FastAPI framework was chosen instead of alternatives such as Django or Flask for several reasons. FastAPI automatically generates interactive Swagger (OpenAPI) documentation, simplifying API testing and usability. It also provides built-in Pydantic-based input validation, ensuring that request data conforms to defined schemas. In addition, FastAPI offers native asynchronous (async) support, enabling efficient handling of concurrent requests while maintaining a concise and readable code structure.

For data storage, SQLite was selected because the project runs on a single-server deployment where high levels of concurrent writes are not required. As a file-based database, SQLite simplifies deployment on platforms such as PythonAnywhere. The system integrates two data sources: the TMDB API for real-time movie information and a Kaggle CSV dataset containing 7,479 films released between 1986 and 2016.

2. System Architecture

The CineMetrics API adopts a modular architecture to improve maintainability and separation of concerns. The application entry point is defined in main.py, which initialises the FastAPI instance and registers all routing modules. Database connectivity is handled in database.py, while relational table structures are defined in models.py using SQLAlchemy. Data validation and request schema enforcement are implemented through Pydantic models in schemas.py, and API key authentication logic is implemented in auth.py. The API functionality is organised into four routing modules—movies, reviews, tags, and analytics—providing a total of sixteen endpoints. Public data retrieval is supported through GET endpoints, whereas write operations (POST, PUT, DELETE) require authentication via the X-API-Key header. Requests without a key return 401 Unauthorized, while invalid keys return 403 Forbidden, ensuring controlled access to protected operations.

3. Testing Approach

The testing strategy was implemented using FastAPI's TestClient to simulate HTTP requests and validate API behaviour. A total of fourteen automated test cases were written in the file test_api.py to verify the correctness of the system. To prevent interference with production data, the tests use a separate SQLite database that is completely isolated from the main application database.

The test suite covers several aspects of the API. These include full CRUD operations for the reviews and tags modules, authentication validation, verification of analytics endpoint responses, and correct handling of 404 errors. Requests without an API key return a 401 response, while invalid keys return 403.

During testing, a challenge was encountered due to the UNIQUE constraint on tmdb_id, which caused conflicts between tests. This issue was resolved by generating a random tmdb_id for each test case.

4. Challenges and Lessons Learned

Several challenges were encountered during the development and deployment of the CineMetrics API. The most significant issue arose from the hosting environment: PythonAnywhere primarily supports WSGI applications, while FastAPI is built on the ASGI standard. Initial attempts to use existing adapters such as a2wsgi and mangum were unsuccessful. The a2wsgi approach caused requests to hang under the

deployment environment, while mangum is designed specifically for AWS Lambda event handling and was therefore incompatible with the PythonAnywhere WSGI workflow. To address this limitation, a lightweight ASGI-to-WSGI adapter was implemented using Python's asyncio.new_event_loop() to execute the FastAPI application within the WSGI request cycle.

Another challenge involved integrating two different data sources: the real-time TMDB API and a static Kaggle dataset. This issue was addressed by introducing a separate service layer responsible for external API interactions and data processing, allowing the routing layer to remain focused on request handling and improving overall system modularity.

5. Limitations and Future Work

The current implementation has several limitations that may affect scalability and functionality. The endpoints GET /reviews/ and GET /tags/ do not currently implement pagination, which may lead to reduced performance when the dataset grows large. This could be improved in future versions by introducing skip and limit query parameters to support efficient pagination. In addition, the system currently uses SQLite as the database engine, which does not handle high levels of concurrent write operations efficiently. A future migration to PostgreSQL would provide better scalability, and the use of SQLAlchemy ORM would make such a transition relatively straightforward. The Kaggle dataset used in this project only covers films released between 1986 and 2016, limiting the historical range of analysis. Future work could integrate more recent datasets to extend analytical capabilities. Finally, the custom WSGI adapter implemented for deployment does not support streaming responses. Migrating the system to a platform with native ASGI support, such as Railway or Render, would allow direct deployment using uvicorn and improve performance and compatibility.

6. Generative AI Declaration

Generative AI tools were used during the development of this project to assist with coding, debugging, and documentation tasks. Specifically, **ChatGPT (OpenAI)** was used to generate initial code templates for the FastAPI application structure, including routing modules, SQLAlchemy ORM models, and Pydantic validation schemas. The tool was also used to assist with debugging issues encountered during development, such as resolving SQLAlchemy UNIQUE constraint errors, diagnosing missing dependencies (e.g., the python-dotenv package) during PythonAnywhere

deployment, and investigating compatibility challenges between WSGI and ASGI frameworks. In addition, ChatGPT was used to support the drafting and editing of this technical report and to improve the clarity and structure of the project README.md documentation. All AI-generated code and suggestions were critically reviewed, understood, and modified by the author before inclusion in the final system. Selected conversation logs are included in the appendix as evidence of AI-assisted development.

7. Appendix: Generative AI Conversation Logs

ChatGPT 5.2

目前我已经想好了我的项目方向，我想以电影API作为我的项目方向。现在我要进行数据的筛查，目前我找到了两种方案，一种是Kaggle 下载数据集，另外一种是接入 TMDB 官方 API，你觉得哪种更好，或者你有什么推荐的吗？

这个问题问得非常专业 👍
你现在已经开始在做"架构级选择"，这一步其实会直接影响你最后能拿多少分。

我们冷静从 评分逻辑 + 技术深度 + 展示效果 + AI使用质量 四个维度分析。

一、两种方案对比

方案 A：使用 Kaggle 数据集（本地导入数据库）

优点

- 数据完全可控
- 可以自己设计数据库 schema
- 可以展示数据清洗过程
- 容易做 analytics endpoint（统计、趋势、评分分布）
- 不依赖外部 API 稳定性
- 适合展示 SQL 能力

缺点

- 数据是"静态"的
- 不能"真实时更动态系统"

已思考 19s

太好了，你的选题（电影 + 影评CRUD + 标签CRUD + 多个分析端点）非常适合冲高分：既能展示 **Web Services（API 设计、鉴权、错误处理、缓存）**，又能展示 **Web Data（数据源筛选、清洗、入库、分析）**，而且口试也很好讲。作业明确鼓励你用公开数据源并展示技术选型与理由 COMP3311_Coursework1_Brief__2026...。

下面我按"先一步一步来"给你结论 + 最稳的落地路线。

1) Kaggle vs TMDB：哪种更好？

只用 Kaggle（静态数据集）

适合： 票房趋势、年代对比、高线统计很稳
风险： 缺少"真实外部服务整合"的工程亮点，容易看起来像数据库作业（但依然能拿 70+，看你架构/测试/鉴权做得怎样）。

只用 TMDB（官方 API）

适合： 电影元数据、类型、演员、海报、热度、搜索、趋势（工程感强）
风险： 你提的"票房趋势"可能会踩坑：TMDB 的 revenue/budget 字段存在**覆盖不全/缺失**的情况（尤其是老片或数据不完整时），如果你把"票房趋势"当主分析，会有数据空洞；另外还要处理 API key 和 rate limit（TMDB 文档提示存在 rate limiting，需要你在设计里体现处理方式）。 The Movie Datab... +2

我给你的推荐（冲高分版本）：Hybrid（强烈建议）

> **TMDB 负责"实时检索 + 元数据"**
> **Kaggle 负责"票房/财务趋势分析的主数扩"**
> **你的数据库负责"影评/标签/推荐所需的用户数据 + 缓存"**

```
.
  app.py
  analytics.py
  services/
    tmdb.py
scripts/
  import_boxoffice.py
tests/
  test_reviews.py
  test_tags.py
README.md
requirements.txt
.env
.gitignore
```

你可以先不写 tests/scripts，但我先把框架留好（后面冲分好加）。

2) requirements.txt（最小可运行）

```
fastapi==0.115.0
uvicorn[standard]==0.30.6
SQLAlchemy==2.0.34
pydantic==2.9.2
pydantic-settings==2.5.2
httpx==0.27.2
python-dotenv==1.0.1
```

sqlalchemy.exc.IntegrityError: (sqlite3.IntegrityError) UNIQU...
FAILED test_api.py::TestAnalytics::test_box_office_trend_returns_200 - assert 404 == 200
FAILED test_api.py::TestAnalytics::test_genre_ratings_returns_list - AssertionError: assert False
========== 8 failed, 4 passed, 5 warnings in 1.27s ==========
(venv) thatcher@thatcherdeMacBook-Air movie-api %

You've got **3 separate issues** here (plus 2 non-fatal warnings). Fix them in this order:

1) `UNIQUE constraint failed: movies.tmdb_id` in `seed_movie()`

What's happening

Your `seed_movie()` always inserts a movie with `tmdb_id=99999`.
But your test DB is **not being reset between tests**, so the second test tries to insert the same `tmdb_id` again → SQLite UNIQUE error.

Fix options (pick 1)

✅ **Option A (best): ensure fresh DB per test via fixture**
Make a fixture that **creates tables + drops tables** for each test (or truncates).

Why You Should Download the Kaggle Dataset

Right now:

- TMDB = working ✅
- Search endpoints = working ✅
- Reviews & Tags CRUD = working ✅
- Basic analytics = partially working ⚠️

But your **box office trend analysis** depends on reliable revenue data.

TMDB revenue fields:

- Often `0`
- Frequently missing
- Not designed for longitudinal financial analysis

If you use TMDB revenue for decade analysis, your professor might ask:

> "How reliable is your financial dataset, and how did you validate it?"

If your answer is "I used TMDB revenue," that's weak.

If your answer is:

> "For financial trend analysis, I used a curated Kaggle box office dataset because it provides structured and normalized revenue values suitable for statistical aggregation."

That's strong.

I downloaded a box office dataset comprising 7,479 films from Kaggle, imported it into the database, and updated the box office trend analysis endpoint. It now returns authentic box office statistics spanning the 1980s to the 2020s.

That's a strong milestone. With Kaggle data integrated and your trend endpoint returning decade-level stats, you've now moved from "API works" to "data-backed analytics," which is exactly what this coursework wants to see.

Here's what you should do next to turn this into a **high-mark submission** (and make the oral exam easy).

1) Sanity-check and strengthen the Kaggle analytics (quick wins, high impact)

Your output looks good, but there's a **red flag**: the 2020s show `movie_count: 15`

```
    loop = asyncio.new_event_loop()
    loop.run_until_complete(app(scope, receive, send))
    loop.close()

    return response_body
```

Yep — this is the source of the problem.

You've written a *home-made* "ASGI inside WSGI" bridge, but it's **not a correct WSGI implementation** (status line is wrong, headers are incomplete, body handling/streaming is wrong, missing required keys in `scope`, and `wsgi.input.read()` will block/consume the stream incorrectly). That's why you're getting weird runtime errors earlier (and it'll keep being flaky even if it "sometimes works").

The fix is simple: **don't hand-roll the adapter**. Use a battle-tested adapter package.

✅ **What to do next (fastest path that works on WSGI)**

Step 1 — Install a proper adapter in your venv

In PythonAnywhere Bash console:

```
cd ~/movie-api
source venv/bin/activate
pip install a2wsgi
```

If I want write the README, Could you provide me with a template?

Absolutely — here is a **clean, coursework-ready README template** tailored to your CineMetrics FastAPI project.
It is structured to look professional, clear, and assessment-friendly.

You can copy this directly into `README.md`.

🎬 CineMetrics – Film Analytics API

CineMetrics is a RESTful film analytics API built using **FastAPI** and **SQLite**.
It integrates real-time film data from the TMDB API and combines it with a Kaggle box office dataset (7,479 films, 1986–2016) to provide analytical insights.

🚀 Features

🚀 Core Functionality

- Film search via TMDB
- Popular/trending films
- Full CRUD operations for:
  - Reviews
  - Tags

please tell me how to PythonAnywhere deployment guide

已思考 43s

Here's a **step-by-step PythonAnywhere deployment guide for your FastAPI (CineMetrics) project**, using PythonAnywhere's **ASGI (FastAPI) beta workflow** (this is the officially documented way to run FastAPI there). PythonAnywhere...

0) Important gotcha (TMDB on free tier)

If you're on a **free PythonAnywhere account**, outbound internet access is restricted to an allowlist. That can break calls to **TMDB** unless it's allowlisted (or you're on a paid plan). See the allowlist page. PythonAnywhere
Paid plans have unrestricted internet access. PythonAnywhere

Practical options:

- **Paid plan (simplest)** → TMDB calls work normally.
- **Free plan** → you may need to (a) request allowlisting, or (b) make TMDB features optional / cached.

```
$ python -m pip install --user --upgrade pip
Collecting pip
  Downloaded pip-25.3-py3.9.egg
Collecting setuptools
  Downloaded setuptools-80.9.0-py3-none-any.whl.metadata (6.6 kB)
Collecting wheel
  Downloaded wheel-0.45.1-py3-none-any.whl.metadata (2.3 kB)
```

I have already configured the environment. What should I do next?

已思考 6s

That screenshot shows you typed `import os` **in the Bash shell**, not in Python. On Linux, `import` is also an **ImageMagick** command, so you got:

`import-im6.q16: unable to open X server ...`

So you're fine — you just ran it in the wrong place.

1) Quick fix: run Python, then import

From your project folder:

```
python -c "import os; print('ok')"
```

or open the Python REPL:

```
Bash
```

1. Technology Stack Justification

2. System Architecture
3. Testing Approach
4. Challenges and Lessons Learned
5. Limitations and Future Work
6. Generative AI Declaration now my report have this 6 part

Your current **6-section structure is actually very good** and already matches what the coursework expects. The brief explicitly says the report should justify the **technology stack, design decisions, testing, challenges, limitations, and AI usage**.
COMP3311_Coursework1_Brief__2026...

You **do not strictly need an "Introduction" section** if your first section (Technology Stack Justification) already explains what the system is. Many technical reports start directly with the design justification.

帮我写一段关于技术栈选择的报告内容，包括以下要点：

选择 Python 是因为它有丰富的数据处理生态（pandas、SQLAlchemy）
选择 FastAPI 而不是 Django 或 Flask，原因是：自动生成 Swagger 文档、Pydantic 输入验证、原生 async 支持、代码简洁
选择 SQLite 是因为单服务器部署不需要并发写入，文件型数据库简化了 PythonAnywhere 部署
两个数据源：TMDB API（实时数据）+ Kaggle CSV（7479 部电影，1986-2016历史数据）
写成学术风格，2-3段，约150词

Technology Stack Justification

Python was selected as the primary programming language due to its extensive ecosystem for data processing and backend development. In particular, libraries such as **pandas** and **SQLAlchemy** provide strong support for data manipulation and relational database interaction, making Python well suited for building data-driven web services. Its readability and large developer community also facilitate rapid development and maintainability. These characteristics make Python a practical choice for implementing an analytics-oriented API such as CineMetrics.

The **FastAPI** framework was chosen instead of alternatives such as **Django** or **Flask** for several reasons. First, FastAPI automatically generates interactive API documentation